Lier des entités avec Doctrine

OneToMany, ManyToOne, ManyToMany…

Table des matières

Lier des entités 2

Accéder aux propriétés 5

Lier des entités

Maintenant qu'on a vu comment créer des Entités, les **peupler**, gérer les routes qui vont y emmener etc, voyons comment on peut lier ces entités entre elles. Je parle ici bien sûr de clés étrangères.

Premièrement, cassons un peu de ce que nous avons fait en **supprimant** la propriété « auteur » de l'entité Livre. On va supprimer la propriété + les getters / setters + La ligne correspondant à l'assignation dans la Fixture.

Puis, nous allons reprendre notre Entité **Livre** grâce au MakerBundle et simplement lui ajouter un champ Auteur. Mais lorsque le MakerBundle va nous demander quel est le type du champ que nous souhaitons ajouter, nous allons lui donner la valeur « **relation** ».

Lançons donc notre commande : `php bin/console make:entity Livre`

Puis nous lui répondons « relation » :

```
Field type (enter ? to see all types) [string]:
> relation
```

Ensuite nous lui pointons la classe « cible » :

```
What class should this entity be related to?:
> Auteur
```

Puis nous allons créer la relation comme étant « ManyToOne ». De plusieurs vers un seul : Plusieurs livres pour un auteur :

```
Relation type? [ManyToOne, OneToMany, ManyToMany, OneToOne]:
> ManyToOne
```

Disons que la propriété ne peut pas être nulle (Un livre sans auteur ça n'existe pas) :

```
Is the Livre.auteur property allowed to be null (nullable)? (yes/no) [yes]:
> no
```

On pourrait croire que ces informations suffisent et effectivement ça pourrait s'arrêter là, ça suffirait. Mais Symfony nous propose de créer cette association… Dans l'autre sens ! Pouvoir aller depuis un auteur vers la liste de ses livres. Nous allons donc accepter volontiers :

```
Do you want to add a new property to Auteur so that you can access/update Livre objects from it - e.g. $auteur->getLivres()? (yes/no) [yes]:
> yes
```

On nous demande enfin quel devra être le nom de la propriété de **l'autre côté** de l'association (le nom généré par défaut est très souvent le bon, mais vérifiez quand même, en français on a des pluriels chiants) :

```
New field name inside Auteur [livres]:
> livres
```

Est-ce que l'on souhaite supprimer les Livres qui se retrouvent sans auteurs ? C'est une bonne question et dans l'absolu on devrait répondre oui, mais déclencher des suppressions automatiques

en cascade est un comportement à utiliser avec parcimonie. Pour l'instant, ne nous embêtons pas avec ces considérations et répondons « no » :

```
Do you want to automatically delete orphaned App\Entity\Livre objects (orphanRemoval)? (yes/no) [no]:
> no
```

On a fini ! Les **deux entités** ont été mises à jour.

Du côté Livre, Doctrine ajoutera une colonne auteur_id dans la table — c'est la clé étrangère SQL.

Du côté Auteur, Collection est une interface Doctrine qui se comporte comme un tableau PHP enrichi. Le constructeur initialise la collection avec un ArrayCollection vide — sans ça, PHP lèverait une erreur à l'instanciation.

inversedBy et mappedBy indiquent à Doctrine quel est le **côté propriétaire** de la relation. Le côté propriétaire est toujours celui qui contient la clé étrangère — ici Livre.

Très intéressant : Dans l'entité Auteur, non seulement on a la propriété et le getter de générés mais on a également addLivre() et removeLivre(). Ces deux méthodes nous permettront d'intéragir directement avec le tableau de livres de l'Auteur, on peut comme si ces deux méthodes constituaient un setter.

Mettons maintenant à jour notre Fixture avant de faire la migration : Si on ne donne pas d'auteurs à nos livres, cela ne pourra pas fonctionner. Voici la nouvelle fonction load() :

```php
public function load(ObjectManager $manager): void
    {
        // On crée les auteurs en premier
        $auteursData = [
            [
                'nom'           => 'Tolkien',
                'prenom'        => 'J.R.R.',
                'nationalite'   => 'Britannique',
                'dateNaissance' => new \DateTimeImmutable('1892-01-03'),
                'biographie'    => 'Romancier et philologue britannique.',
            ],
            [
                'nom'           => 'Herbert',
                'prenom'        => 'Frank',
                'nationalite'   => 'Américaine',
                'dateNaissance' => new \DateTimeImmutable('1920-10-08'),
                'biographie'    => 'Auteur de science-fiction américain.',
            ],
            [
                'nom'           => 'Orwell',
                'prenom'        => 'George',
                'nationalite'   => 'Britannique',
                'dateNaissance' => new \DateTimeImmutable('1903-06-25'),
                'biographie'    => 'Romancier et essayiste britannique.',
            ],
        ];
```

```
        foreach ($auteursData as $data) {
            $auteur = new Auteur();
            $auteur->setNom($data['nom']);
            $auteur->setPrenom($data['prenom']);
            $auteur->setNationalite($data['nationalite']);
            $auteur->setDateNaissance($data['dateNaissance']);
            $auteur->setBiographie($data['biographie']);

            $manager->persist($auteur);

            // On stocke une référence pour pouvoir la réutiliser plus bas
            $this->addReference('auteur-' . strtolower($data['nom']), $auteur);
        }

        // Ensuite les livres, en récupérant les références des auteurs
        $livresData = [
            [
                'titre'  => 'Le Seigneur des Anneaux',
                'annee'  => 1954,
                'genre'  => 'Fantasy',
                'resume' => 'Un hobbit part à l\'aventure pour détruire un anneau
maléfique.',
                'auteur' => 'tolkien',
            ],
            [
                'titre'  => 'Dune',
                'annee'  => 1965,
                'genre'  => 'Science-Fiction',
                'resume' => 'Sur une planète désertique, un jeune noble affronte
son destin.',
                'auteur' => 'herbert',
            ],
            [
                'titre'  => '1984',
                'annee'  => 1949,
                'genre'  => 'Dystopie',
                'resume' => 'Dans un monde totalitaire, un homme tente de résister
au système.',
                'auteur' => 'orwell',
            ],
        ];

        foreach ($livresData as $data) {
            $livre = new Livre();
            $livre->setTitre($data['titre']);
            $livre->setAnnee($data['annee']);
            $livre->setGenre($data['genre']);
            $livre->setResume($data['resume']);
            $livre->setAuteur($this->getReference('auteur-' . $data['auteur'],
Auteur::class));

            $manager->persist($livre);
        }

        $manager->flush();
    }
```

On passe une **référence** à l'objet Auteur dans la propriété **$auteur** de Livre

Il ne nous reste plus qu'à « commit » ces modifications de la structure de la base, en demandant à Doctrine d'effectuer une migration.

Mais vu que pour l'instant, on n'a toujours pas d'auteurs en base, la génération de la clé étrangère va provoquer une erreur. En effet, ajouter le champ auteur_id dans la table livre sans avoir d'auteurs en base, ça va forcément le mettre à NULL, et ça ce n'est pas possible. On va donc écraser notre base et la refaire, mais ça tombe bien Doctrine + les Fixtures sont faits pour ça

- On vide la base : `php bin/console doctrine:database:drop --force`
- On la recrée : `php bin/console doctrine:database:create`
- D'abord on génère le script de migration : `php bin/console make:migration`
- Puis on l'exécute : `php bin/console doctrine:migrations:migrate`
- Et enfin on charge la Fixture : `php bin/console doctrine:fixtures:load`

Accéder aux propriétés

Maintenant que Livre a un Auteur, on peut afficher son nom directement dans les templates grâce au chaînage de propriétés de Twig :

```
{% for livre in livres %}
    <div>
        <h2>{{ livre.titre }}</h2>
        <p>{{ livre.auteur.prenom }} {{ livre.auteur.nom }} – {{ livre.annee }}</p>
        <p><em>{{ livre.genre }}</em></p>
        <a href="{{ path('app_livre_detail', { id: livre.id }) }}">
            Voir le détail
        </a>
    </div>
{% endfor %}
```

Concrètement ici, on accède à la propriété **prenom** de la propriété **auteur** de notre livre. On appelle cela le chaînage de propriétés.

EXERCICE

1. Utilisez la propriété « livres » de la classe Auteur pour afficher la liste des livres d'un auteur sur la page détail d'un auteur. Pour l'instant, vous n'aurez qu'un livre, c'est normal, mais si vous vous sentez d'en rajoutez, vous savez où ça se passe maintenant -> Fixture
2. Toujours dans la vue détail de l'auteur, pour chaque livre affiché, générer un lien permettant d'aller consulter la page détail du livre en question
3. Stylisez un peu tout ça, ça sera fait puisque nous allons continuer sur ce sujet encore quelques temps ☺ Appropriez-vous le framework !